

Defending Network Intrusion Detection Systems Based on Graph Neural Networks Against Structural Adversarial Attacks

Dimitri Galli*, Andrea Venturi[†], Dario Stabili*, Mauro Andreolini*, Mirco Marchetti*

*Department of Engineering “Enzo Ferrari”, University of Modena and Reggio Emilia, Modena, Italy
{dimitri.galli, dario.stabili, mauro.andreolini, mirco.marchetti}@unimore.it

[†]Department of Digital Security, Vicomtech, San Sebastián, Spain
aventuri@vicomtech.org

Abstract—Graph Neural Networks (GNNs) represent a promising solution for Machine Learning (ML) based Network Intrusion Detection Systems (NIDS), thanks to their ability to leverage both network flow features and topological patterns. While GNN classifiers demonstrate superior robustness against feature-based adversarial attacks compared to other ML detectors, they remain vulnerable to structural adversarial attacks, where an attacker perturbs the underlying network graph topology by injecting edges or inserting nodes. Such attacks pose a realistic and severe threat, undermining the reliability of GNN-based NIDS in practical deployments. While countermeasures have been proposed in the literature, they often rely on assumptions that are unrealistic in real-world cybersecurity scenarios. In this paper, we propose a defense framework based on adversarial training to strengthen GNN-based NIDS against structural attacks. We generate adversarial samples by strategically replacing the source and destination nodes in benign network flows, thereby efficiently mimicking edge injection attacks. We evaluate our approach on two widely used datasets (CTU-13 and TON-IoT) using E-GraphSAGE as the base GNN classifier. Experimental results show that our approach produces hardened detectors with superior detection performance on clean graphs and enhanced robustness against structural adversarial attacks.

Index Terms—adversarial training, graph neural networks, machine learning, network intrusion detection systems, structural attacks

I. INTRODUCTION

The exponential growth of network-based cyberattacks has increased the demand for accurate and adaptive Network Intrusion Detection Systems (NIDS) [1], [2]. In recent years, Machine Learning (ML) has emerged as a powerful paradigm for building these systems, enabling the automatic discovery of malicious patterns in network traffic and improving the detection of unknown attacks [3].

Despite these advances, ML-based NIDS remain highly vulnerable to *adversarial ML attacks* [4]. Such attacks carefully perturb input features to cause misclassifications, enabling malicious traffic to evade detection mechanisms. These manipulations provide attackers with a powerful means to conceal their activities while preserving the intended impact of their

attacks [5]. The effectiveness of these attacks represents a concrete threat to the reliable deployment of ML-based NIDS in real-world environments.

Graph Neural Networks (GNNs) have emerged as a promising alternative for ML-based NIDS [6] due to the inherent graph-like structure of network traffic. GNN-based NIDS take network traffic in graph form as input data, modeling internal hosts as nodes and communication flows between them as edges. By analyzing both flow-level information (e.g., duration, byte count) and network topology, GNN-based NIDS have demonstrated superior detection performance compared to traditional ML detector [7], [8]. Empirical studies also show that GNNs exhibit stronger robustness against feature-based adversarial attacks due to their reliance on structural attributes [9].

However, incorporating network topology in the decision process can expose classifiers to a new class of threats known as *structural adversarial attacks*. These attacks manipulate the graph topology to induce misclassifications [10]. Since GNNs heavily rely on neighborhood structures, even small perturbations can dramatically degrade their detection performance. This is particularly concerning in cybersecurity scenarios, as such attacks can be launched by compromised hosts generating seemingly legitimate, and therefore hard-to-detect, network traffic [11]. While the scientific literature has proposed defenses against structural adversarial attacks, only a few have been explored in the cybersecurity domain. Moreover, many existing methods depend on prior knowledge of attack techniques or require computationally expensive resources, resulting in high latency and limiting their real-time deployment [12], [13].

Among traditional adversarial ML defenses, *adversarial training* stands out as a promising approach to enhancing classifier resilience by retraining ML models on an augmented training set that includes carefully crafted adversarial examples [14]. Nevertheless, its effectiveness in the specific context of GNN-based NIDS remains largely unexplored in prior literature. This work aims to fill this gap by introducing a new defensive method based on adversarial training. The key contribution lies in a novel strategy for generating ad-

versarial examples, which considers the unique characteristics of the network intrusion detection domain and can be used to harden GNN-based NIDS against structural adversarial attacks. Specifically, we propose to generate synthetic adversarial flows by modifying both the source and destination endpoints of benign netflows with *low-degree nodes*. These nodes with few connections are structurally more vulnerable and therefore attractive to real-world attackers seeking to evade detection. Each modified record corresponds to a newly injected benign edge connecting low-degree nodes, thereby perturbing the graph topology.

We apply our approach to an extensive experimental campaign using two popular datasets of enterprise network flows. Our method targets various types of structural adversarial attacks, including those involving edge additions, node additions, or combinations of both. The results demonstrate the effectiveness of the proposed strategy, with absolute performance improvements of up to 90% in the best case against structural perturbations. Moreover, the hardened classifiers maintain high detection performance even in the absence of adversarial attacks.

The remainder of this paper is organized as follows. Section II provides background on ML-based NIDS, GNN-based NIDS, as well as adversarial ML attacks and corresponding defense mechanisms. Section III defines the threat model. Section IV describes the proposed defense strategy. Section V details the testbed. Section VI presents and discusses the experimental results. Section VII concludes the paper with final remarks.

II. BACKGROUND AND RELATED WORK

This section provides background on NIDS based on ML and GNNs. We then motivate our work by examining the threat posed by adversarial attacks and identifying critical gaps in the adversarial defense literature.

A. ML-based NIDS

ML-based NIDS analyze and classify network data to detect malicious operations, either by processing raw packets directly [15] or by summarizing traffic into network flows¹ (or *netflows*) using selected metrics and statistics [16]. Depending on the approach, these systems may rely on labeled data for supervised training or operate in an unsupervised manner, learning normal behavior patterns without explicit labels [3].

Traditional ML classifiers often consider each sample in isolation. While this enables rapid processing and the real-time generation of security alerts, analyzing records individually limits the possibility of detecting complex multi-step attacks, which require capturing relational dependencies across multiple connections and hosts [9], [17].

B. GNN-based NIDS

Graph Neural Networks [18] have recently emerged as a compelling alternative for cyber threat detection, as they can naturally represent network communication patterns as

graph structures [6]. GNNs are deep learning models specifically designed to handle graph-structured data, leveraging the premise that cyberattacks exhibit distinctive topological patterns detectable through network structure analysis.

Early research has shown that GNN-based NIDS outperform traditional ML detectors both in adversarial environments [9], [19] and in attack-free settings [7], [8]. This performance stems from the GNN's ability to simultaneously process netflow attributes and structural features, resulting in more robust detection capabilities [20].

The operational workflow of a GNN-based NIDS involves three key phases. The first phase builds the *graph* from netflow data. The most common representation, known as a *flow graph*, maps each unique endpoint to a graph node, while individual netflows are represented as edges connecting the corresponding vertices. This graph representation allows the detection problem to be formulated as an edge classification task [7]. However, the majority of GNN models are designed for node-level tasks, so alternative representations have been studied to analyze node-associated features during training. For instance, a *line graph* maps individual netflows directly to nodes and connects them when they share a common host in the corresponding records [8]. Once netflows are transformed into an appropriate graph structure, the GNN is trained to generate *node embeddings* that capture higher-order dependencies through message passing and neighborhood aggregation. Such embeddings encode both netflow information and network topology in a low-dimensional latent space. *Edge embeddings* can be derived by concatenating the latent vectors of the respective source and destination nodes [7]. To perform detection, these embeddings are fed into a *classifier* that distinguishes benign from malicious network traffic.

C. Adversarial Attacks

Adversarial ML attacks exploit a classifier's sensitivity to carefully crafted perturbations that mislead the model, causing it to produce incorrect outputs that benefit the attacker [21], [22]. These attacks can occur during the training phase, as in *poisoning attacks*, or inference, as in *evasion attacks*, depending on whether the attacker aims to compromise the ML model before or after deployment.

In network intrusion detection scenarios, adversaries typically manipulate common network traffic attributes (e.g., duration, byte count, packet size) to deceive the ML detector and induce misclassifications [23]. As GNN-based NIDS rely on both flow features and structural patterns, attackers can evade detection by targeting either netflow information (*feature attacks*) or the graph topology (*structural attacks*) [10].

On the one hand, several studies have demonstrated that GNN-based NIDS are relatively robust to feature attacks due to their reliance on neighborhood structures [9], [11], [19], [24]. As a result, feature perturbations pose a comparatively lower risk to these systems. On the other hand, structural attacks targeting the graph topology represent a greater threat to GNN-based NIDS. These attacks can severely disrupt GNN performance, with minimal perturbations sometimes enabling

¹<https://www.cisco.com/c/en/us/products/ios-nx-os-software/ios-netflow/>

near-complete evasion while affecting only a small fraction of the original graph [10], [11], [25].

D. Adversarial Defenses

Although various countermeasure strategies have been proposed to defend GNNs against such attacks, only a few have been specifically designed or evaluated in the context of network intrusion detection. This scarcity of practical defenses highlights a critical gap, as most existing solutions target general graph learning tasks and may not account for the unique characteristics and constraints of NIDS environments.

Preprocessing-based defenses [26], [27] sanitize the graph by detecting and removing adversarial manipulations before the input is submitted to the GNN. While effective against specific structural perturbations (e.g., *edge deletion attacks* [28]), such defenses often depend on prior knowledge of attack strategies, which is rarely available in real-world networks. These defenses may also inadvertently alter or remove legitimate graph components (e.g., benign edges) that are essential for modeling the network topology. This can obscure important structural relationships, making it harder for the NIDS to detect both known and novel attack patterns.

Model-based defenses modify GNN architectures to improve their resilience to perturbations. Defenses such as *defensive aggregation* [29] or *robust message passing* [30] are designed to mitigate the effect of adversarial modifications on model predictions. However, these methods typically introduce significant computational overhead, resulting in excessive latency that can delay the detection of malicious activity. Moreover, sophisticated adaptive attacks can still exploit specialized architectures, highlighting the persistent arms race between attackers and defenders.

Training-based defenses [31], [32] enhance GNN robustness against evasion attacks by leveraging specialized training strategies. In particular, *adversarial training* [33] aims to harden the classifier by incorporating carefully perturbed samples into the original training set, effectively reshaping the model’s decision boundary to enable accurate classification of adversarial patterns. However, adversarial training faces several challenges when applied to network intrusion detection tasks. Compared to other domains, adversarial samples are bound by strict validity requirements that preserve protocol compliance, feature consistency, and temporal dependencies [5]. Indeed, adversarial training is generally effective only against predictable attacks that resemble the generated examples, which can lead to a false sense of robustness [34]. Although it can enhance robustness against specific threats, it often reduces performance on clean data, thereby limiting its practicality in real-world scenarios [35].

Our work builds on adversarial training to strengthen GNN-based NIDS against structural attacks. Unlike model-based defenses, it directly operates on the netflows in the training set without altering the classifier architecture. Notably, we generate adversarial examples by modifying the topology information of benign records, without removing legitimate components. Since we apply perturbations at the structural

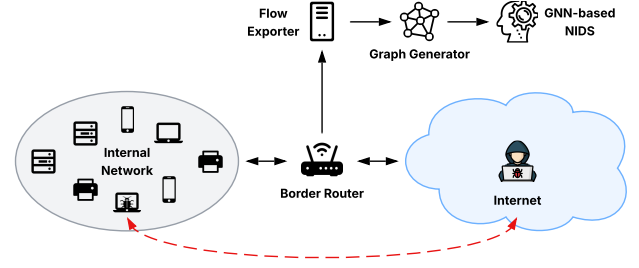


Fig. 1: Threat model.

level, we preserve original flow features, ensuring that all network constraints are maintained.

III. THREAT MODEL

In this section, we define the *threat model* for this work. More specifically, we outline the goals, knowledge, and capabilities of both the attacker (Section III-A) and the defender (Section III-B).

Building upon prior work [36], we consider a traditional network intrusion detection scenario, as depicted in Fig. 1. A medium-sized network connects to the Internet through a *border router*, which handles all incoming and outgoing traffic for internal hosts. The border router also forwards network packets to a *flow exporter*, which collects and extracts flow records. Once the netflows are produced, a *graph generator* builds a graph representation, which is then used as input to the GNN classifier. The GNN-based NIDS consists of an *ensemble* of M graph-based individual models. Each model is trained to identify suspicious activities corresponding to one of the M distinct malicious classes.

A. Attacker Model

The attacker model employed to evaluate the effectiveness of our proposed defense reflects a realistic NIDS threat scenario and builds upon prior work on adversarial attacks in the problem space [5], [11]. We consider an attacker located outside the monitored network who has already compromised one host within the enterprise network via common attack vectors, such as software vulnerabilities or weak authentication (see red arrow in Fig. 1).

The attacker’s primary *goal* is to remain undetected while carrying out malicious operations within the organization’s network. The attacker seeks to evade detection by inducing the NIDS to misclassify malicious communications, all while ensuring that the underlying attacks retain their full effectiveness.

The attacker operates under a gray-box setting with limited *knowledge* of the target system. In particular, the attacker assumes that a GNN-based NIDS is deployed but lacks detailed information about the classifier configuration, feature set, and graph representation. The attacker has no access to the training data and operates only at inference time. Notably, the attacker cannot observe the predictions of the targeted GNN detectors.

The attacker’s *capabilities* are defined by the actions they can perform on the network itself. While the attacker has complete control over a limited number of compromised hosts, they cannot directly interact with core infrastructure components such as the flow exporter, graph generator, or NIDS. The attacker cannot manipulate the features of the graph representation submitted to the NIDS (i.e., no feature space perturbations). Their capabilities are therefore restricted to the following threats:

- **$C2x_B$ attacks:** injecting *benign* edges (denoted by B) into the graph by establishing legitimate communications from *compromised* hosts (denoted by C) to *random* ones (denoted by x) within the network;
- **$C2x_M$ attacks:** injecting *malicious* edges (denoted by M) into the graph by initiating unauthorized communications from *compromised* hosts (denoted by C) to *random* ones (denoted by x) within the network;
- **Add node attacks:** injecting *new* nodes into the graph by communicating with external devices not previously part of the network or by compromising internal hosts.

B. Defender Model

We consider the defender to be the network administrator responsible for deploying, maintaining, and monitoring the NIDS within the organization, or alternatively, the security vendor that develops and tunes it. The defender’s primary *goal* is to enhance the resilience of GNN-based NIDS against adversarial attacks based on structural perturbations. In other words, the defender aims to build a defense mechanism that enables GNN classifiers to detect malicious traffic even in the presence of attackers executing topological manipulations within the network graph. Moreover, the defender seeks to preserve or improve the NIDS’s original detection accuracy on unperturbed input.

The defender has complete *knowledge* of the underlying network infrastructure, as well as the internal configuration and operational details of the GNN-based NIDS, such as feature space, graph representation, and model parameters. The defender has access to historical malicious traffic that previously triggered security alerts and was confirmed as part of verified security incidents. However, the defender lacks prior knowledge of the adversary’s specific strategies or technical implementations, reflecting the practical challenge of defending against unknown attack variants. Furthermore, the defender does not know exactly which hosts have been compromised by the attacker and thus cannot rely on knowledge of malicious nodes for their defense method.

The defender maintains full control over all machines within the private network. In line with their role as system and network administrators, the defender’s *capabilities* include:

- Access historical network traffic data and corresponding labels (i.e., the training dataset);
- Initiate legitimate communications and replace the endpoints involved in the transmission, thereby generating adversarial examples;

- Expand the training set with the adversarial flows and modify the GNN classifier’s training process.

IV. PROPOSED DEFENSE STRATEGY

In this section, we describe the proposed defense framework to harden GNN-based NIDS against structural adversarial attacks. As mentioned in Section II, our defense methodology is based on adversarial training. Indeed, adversarial training is widely recognized as one of the most effective strategies for defending ML classifiers against adversarial attacks [37], [38]. Building on early work in other fields that demonstrates its effectiveness for GNN models [33], we design and revisit this approach to enhance the robustness of GNN-based NIDS.

An overview of the proposed two-stage defense strategy is illustrated in Fig. 2. The considered example network contains four unique hosts indicated by the letters A , B , C , and D . Each of the nodes C and D has only one incoming edge, giving it a *degree* equal to 1. The GNN-based NIDS operates on flow graphs, where nodes represent network hosts and edges store the features of the respective netflows. Even if our discussion focuses on flow graphs, the proposed methodology can be extended to other graph representations (e.g., line graphs). As noted in Section II, features associated with edges can be directly translated to vertices.

As illustrated at the top of Fig. 2, the first part of our methodology involves a *warm-up phase*. In this phase, the GNN classifier is trained exclusively on clean network data. This part allows the model to learn the fundamental patterns of benign and malicious network communications. The warm-up is essential for establishing stable decision boundaries that can later be refined using adversarial data [39].

The bottom of Fig. 2 depicts the *hardening phase* of our strategy. Unlike traditional adversarial training, our approach does not restart training from scratch; instead, it continues from the warm-up stage. Since the nodes C and D are the least connected vertices in the graph, they are employed to replace both the original source and destination hosts in benign communications, generating adversarial structural flows. Specifically, we iteratively sample subsets of increasing size from the benign netflows in the initial training set and perturb them by replacing their original endpoints with low-degree nodes, i.e., those having a degree below the first percentile of the degree distribution. During this procedure, the training set is augmented with an increasing number of perturbed samples. This process effectively injects new benign edges between reassigned nodes in the resulting graph, exposing the GNN detector to controlled structural noise.

In the following, a detailed explanation of the two phases.

A. Warming Up

The *warm-up phase* consists of an initial training procedure where the GNN-based NIDS is exposed to untouched network data. This phase defines the classifier’s baseline capabilities on clean data before adversarial examples are injected into the training set. The goal is to regularize the learning process by establishing decision boundaries that separate benign and

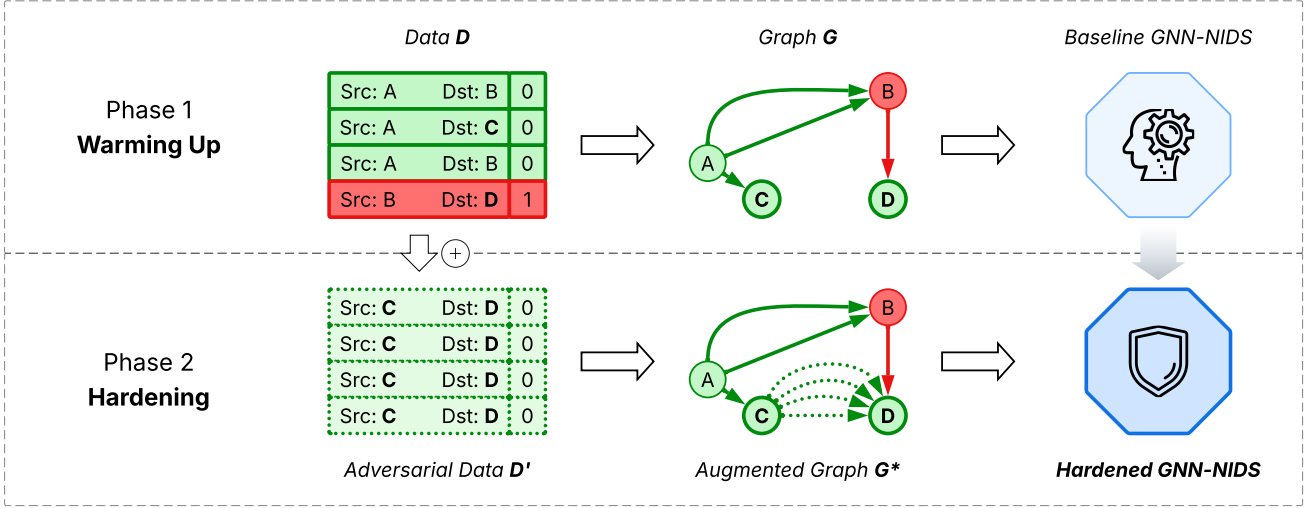


Fig. 2: Defense strategy to counter structural adversarial attacks against GNN-based NIDS.

malicious traffic. Indeed, premature exposure to adversarial samples could destabilize gradient updates and hinder loss convergence [39].

We denote the original training set as $\mathcal{D} = \{(f_i, y_i)\}_{i=1}^N$, where f_i is the i -th flow example and y_i is its ground-truth label. A netflow is considered *benign* if $y_i = 0$, and *malicious* if $y_i = 1$. Each flow record f_i contains both structural information (i.e., *endpoints*) and traffic statistics (i.e., *features*). In particular, each netflow f_i is defined by a tuple (s_i, d_i, x_i) , where s_i denotes the *source endpoint* (e.g., 109.80.75.71 : 36457), d_i the *destination endpoint* (e.g., 147.32.84.188 : 8080), and x_i the *feature vector* (e.g., *duration, bytes, packets*).

Based on the training set $\mathcal{D}$, we build a flow graph $G = (V, E)$, where V is the set of nodes corresponding to unique network endpoints, and E is the set of edges corresponding to flow examples. More specifically, each netflow f_i generates an edge between nodes s_i and d_i , attributed with the feature vector x_i and the binary label y_i .

During warm-up, we train the GNN classifier on the clean flow graph G for a fixed number of warm-up epochs. The GNN parameters are optimized by minimizing a supervised loss function, enabling the model to encode stable representations of benign and malicious samples.

B. Hardening

After the warm-up phase, the GNN-based NIDS transitions into the *hardening phase*, which constitutes the core of our defense approach. In this stage, the training set is progressively expanded with manipulated flows that induce novel edges within the flow graph. This augmentation prevents the classifier from being overwhelmed at the beginning of training. Instead, the number of perturbed samples is gradually increased to enable the classifier to refine its decision boundaries.

To generate adversarial examples, we randomly sample a subset $\mathcal{B}_t$ from the benign flows $\mathcal{B}$. The total amount of benign netflows sampled for manipulation is given by $\epsilon_t \cdot |\mathcal{B}_t|$, where ϵ_t is the perturbation intensity and $|\mathcal{B}_t|$ is the number of selected records. Hence, the parameter ϵ_t controls the number of perturbed samples injected into the original training set. The value of ϵ_t depends on epoch t and is progressively increased every T epochs during the hardening stage. While the perturbation intensities can be chosen freely, we bound these values to maintain realistic network conditions and prevent excessive distortion of the original traffic patterns. Indeed, we observe that large perturbations offer no advantage to the defender and may harm clean performance in non-adversarial scenarios [40].

For each sampled benign flow $f_i = (s_i, d_i, x_i) \in \mathcal{B}_t$, we generate its adversarial version $f'_i = (s'_i, d'_i, x_i) \in \mathcal{B}'_t$ by replacing both the source and destination endpoints (s_i, d_i) with different endpoints (s'_i, d'_i) . The feature vector x_i remains unchanged, preserving original traffic attributes [5]. We select as new endpoints (s'_i, d'_i) the *low-degree nodes* V' of the graph obtained from the training set. More specifically, we identify a pool of nodes whose connectivity falls below the first percentile of the total degree distribution of the original network graph and use it to perturb the structural information of the sampled netflows $\mathcal{B}_t$. This operation simulates a practical *edge injection attack*, establishing new communications from low-degree nodes to random vertices. In fact, the edges in the network graph are relocated when we replace the endpoints of the pool of benign samples: (1) the edge between the initial source and destination nodes (s_i, d_i) is dropped; (2) the feature vector x_i is saved; (3) a new edge with the same features x_i is set up between the replacement hosts (s'_i, d'_i) .

Finally, we combine the original examples f_i with the newly generated flows f'_i to form the augmented dataset $\mathcal{D}^*$. This

expanded dataset $\mathcal{D}^*$ is then converted into a graph G^* , which serves to train the GNN using the supervised loss and model weights obtained at the end of the warm-up.

V. TESTBED

This section presents the experimental campaign conducted to validate the effectiveness of the proposed defense. Section V-A describes the datasets employed, Section V-B details the GNN-based detectors, and Section V-C outlines the implementation of our framework. To promote reproducibility and transparency, we publicly release our code and experimental results².

A. Datasets

We adopt two publicly available datasets: **CTU-13** [41] and **TON-IoT** [42]. These two datasets are well-known benchmarks in the intrusion detection domain and are widely referenced in the literature [8], [43]. They are particularly suitable for our task, as they provide labeled netflows that can be readily transformed into graph-based representations [7], [8]. Together, both datasets encompass a broad spectrum of cyberattacks and structural patterns, making them ideal for evaluating the robustness of GNN-based NIDS.

On the one hand, CTU-13 comprises thirteen scenarios, each consisting of several hours of traffic from a medium-sized enterprise network. Together with benign activity, each scenario includes malicious traffic generated by a specific botnet variant. On the other hand, TON-IoT includes heterogeneous data collected from IoT environments, such as system logs, raw telemetry, and network traces. In this work, we use only the flow records, which include multiple types of cyberattacks.

We preprocess both datasets following the methodology proposed in [44]. The resulting feature set includes source and destination endpoints, with more than 30 statistical attributes, consistent with prior studies [12], [43]. Benign flows $\mathcal{B}$ are merged into a single dataset, whereas malicious flows $\mathcal{M}$ associated with each threat α are grouped into dedicated subsets $\mathcal{M}_\alpha$. We discard malicious variants with too few records.

For each threat α , we construct a dataset $\mathcal{D}_\alpha$ by combining benign flows $\mathcal{B}$ with malicious samples $\mathcal{M}_\alpha$, using a benign-to-malicious ratio of 10 : 1 to reflect the imbalance typically observed in operational networks. Each dataset $\mathcal{D}_\alpha$ is divided into training and test sets with an 80 : 20 split.

Table I reports the resulting dataset statistics, i.e., the amount of malicious netflows together with the number of nodes and edges of the generated graphs. Each row corresponds to a distinct malicious variant α .

B. Detector

In our experimental campaign, we target a NIDS based on the **E-GraphSAGE** [7] architecture. It is an extension of the popular GraphSAGE model [45] that performs edge classification for network intrusion detection.

TABLE I: CTU-13 and TON-IoT datasets statistics.

Dataset	Threat	Training			Testing		
		#Mal	#Nodes	#Edges	#Mal	#Nodes	#Edges
CTU-13	Neris	64 076	540 679	1 409 672	16 019	167 881	352 418
	Rbot	22 007	228 374	484 158	5 502	68 611	121 040
	Virut	25 875	252 747	569 254	6 469	75 734	142 314
	Menti	2 260	29 327	49 720	565	8 339	12 430
	Murlo	885	12 341	19 464	221	3 392	4 868
TON-IoT	Bkdr	10 522	27 449	231 496	2 631	9 173	57 874
	DDoS	10 522	36 683	231 496	2 631	11 209	57 874
	DoS	10 522	28 188	231 496	2 631	9 362	57 874
	Inj	10 522	36 265	231 496	2 631	11 169	57 874
	Pswd	10 522	36 356	231 496	2 631	11 191	57 874
	Rans	10 522	28 608	231 496	2 631	9 680	57 874
	Scan	10 522	32 089	231 496	2 631	10 753	57 874
	XSS	10 522	33 874	231 496	2 631	10 960	57 874

E-GraphSAGE converts network traffic into a flow graph, where each unique endpoint is represented as a node, and each netflow corresponds to an edge linking the endpoints included in it. Each edge is attributed with the record's statistical features (e.g., duration, byte count, packet size).

The model processes the input through multiple graph convolutional layers to generate low-dimensional vectors for each edge. These embeddings encode both the edge's features and information from its local neighborhood. Importantly, the inductive nature of E-GraphSAGE allows the generation of embeddings for previously unseen nodes at inference time. This is a key property for a GNN-based NIDS operating in dynamic and realistic network environments.

Following best practices [3], the detector is organized as an ensemble of binary classifiers, each implemented as a specialized E-GraphSAGE model trained to detect a specific malicious class α . All classifiers are configured with the same hyperparameters as in the original work [7] to ensure consistency.

C. Framework Implementation

We now describe in detail the implementation of our framework (Section IV). All experiments were implemented in *Python*, using *pandas* and *scikit-learn* for data preprocessing and graph generation, and *DGL* with *PyTorch Geometric* for GNN training and testing.

As outlined in Section V-A, each dataset $\mathcal{D}_\alpha$ is split into training and test sets using an 80 : 20 ratio while preserving a 10 : 1 benign-to-malicious class distribution. First, we obtain clean flow graphs from the training data and use them to train a *baseline* NIDS composed of an ensemble of E-GraphSAGE classifiers, each specialized in detecting a specific malicious variant α (Section V-B).

We also train a hardened version of each classifier for comparison. As discussed in Section IV-A, each E-GraphSAGE classifier undergoes a warm-up phase of 100 epochs on the unperturbed flow graph to establish stable decision boundaries. The learning process then continues for an additional 100 epochs on adversarially augmented data, starting from the loss value and model weights obtained at the end of the warm-up. As explained in Section IV-B, we collect and perturb progressively larger sets of benign flows. The number of benign samples selected for manipulation at epoch t is controlled by

²<https://github.com/dimgalli/defending-gnn-nids.git>

the perturbation intensity ϵ_t , which starts at 1 and increases by 1 every $T = 20$ epochs up to a maximum of 5. We cap ϵ_t at 5 to ensure realistic structural conditions and avoid excessive distortion of decision boundaries that could degrade clean performance.

At each step, benign records $\mathcal{B}_t$ are modified by replacing their source and destination endpoints with low-degree nodes. The perturbed samples $\mathcal{B}'_t$ are injected into the training set to generate the adversarially augmented dataset $\mathcal{D}^*$, which is transformed into the expanded graph G^* that can be used to harden E-GraphSAGE models.

VI. EXPERIMENTAL RESULTS

This section presents the results of our experimental campaign, which aims to prove two key points: (1) our defense yields a GNN-based NIDS whose performance on clean network graphs remains comparable or superior to state-of-the-art models and (2) the proposed method significantly enhances resilience against structural adversarial attacks.

We compare the detection performance of the baseline and hardened models across two different evaluation scenarios: *clean performance* on unperturbed data (Section VI-A) and *adversarial robustness* against structural attacks (Section VI-B).

In the first scenario, we evaluate the detection accuracy of both classifiers on clean inputs. We target both classifiers with test flow graphs that do not involve any structural manipulation. These experiments verify that both classifiers correctly detect the malicious variants they were trained on without worsening clean performance under standard operating conditions. This constraint is critical, as a NIDS that performs poorly on known attacks is unsuitable for practical deployment in real-world networks.

In the second scenario, we measure the adversarial robustness of both detectors against realistic structural attacks. We employ the **C2x_B**, **C2x_M**, and **add node** structural attacks proposed in [11] for this evaluation. These attacks comply with real-world constraints and have demonstrated high effectiveness against state-of-the-art GNN detectors. Hence, they serve as a representative benchmark for evaluating robustness in adversarial settings.

For evaluating the classifiers on clean data, we report performance using standard ML metrics from the literature, i.e., *F1-score*, *Precision*, and *Recall (Detection Rate)*. These metrics are defined as follows:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (1)$$

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

where *TP*, *FP*, and *FN* denote *true positives*, *false positives*, and *false negatives*, respectively. For the evaluation of the adversarial robustness, we use the *Detection Rate (DR)*. In the paper, we consider a positive detection as a malicious sample.

A. Clean Performance

We first evaluate the performance of the baseline and hardened E-GraphSAGE classifiers on clean flow graphs.

We report the results in Table II: each row compares the scores of both classifiers for a specific malicious activity, while the columns correspond to the performance metrics. For each class, we highlight the best scores in bold. For each dataset, the bottom row presents the mean (μ) and standard deviation (σ) of the metrics values across all models.

The results show that nearly all values exceed 0.9, confirming the strong detection capabilities of E-GraphSAGE. The only isolated case concerns the *Neris* botnet of the *CTU-13* dataset, where the baseline and hardened detectors yield slightly lower *F1-scores* of 0.880 and 0.891, respectively. In most cases, the hardened classifiers outperform their baseline counterparts, achieving average *F1-scores* of 0.962 versus 0.959 on *CTU-13*, and 0.995 versus 0.994 on *TON-IoT*.

Overall, E-GraphSAGE models demonstrate high performance consistent with state-of-the-art GNN-based NIDS [7], [12]. These findings endorse that our defense framework maintains good performance under clean conditions, slightly improving *F1-scores*, even when *Precision* experiences minor decreases. The strong performance of E-GraphSAGE on unperturbed data indicates that the hardened NIDS is suitable for real-world deployment and motivates its further evaluation under adversarial conditions (Section VI-B).

B. Adversarial Robustness

We now evaluate the resilience of the baseline and hardened E-GraphSAGE classifiers against adversarial attacks involving structural perturbations. Building on previous works [11], attackers can perturb the test flow graph in three distinct ways:

- **C2x_B attacks**: compromised hosts establish benign communication to random targets;
- **C2x_M attacks**: controlled machines initiate malicious connections to random targets;
- **Add node attacks**: new, previously unseen endpoints generate benign flows.

This set of attacks represents the main perturbation strategies available to a capable attacker. The goal is to determine whether our adversarial training approach enhances model robustness under such conditions.

The results are shown in Fig. 3, with separate subfigures for each attack type: Fig. 3a for **C2x_B attacks**, Fig. 3b for **C2x_M attacks**, and Fig. 3c for **add node attacks**. Each plot reports the *Detection Rate (DR)* (y-axis) of both classifiers as the *perturbation step* (x-axis) increases. For **C2x_B** and **C2x_M** attacks, the perturbation step indicates the number of perturbed flows injected into the test set (i.e., added edges from compromised nodes to random ones), with values of 0, 1, 2, 5, 10, and 20. For **add node** attacks, it represents the number of new nodes injected into the resulting test graph, with values of 0, 1, 5, 10, 100, and 1000. We have the same model performance on clean data (Section VI-A) with a perturbation step of 0. Each curve in the plots represents the robustness trend (in terms of *DR*) for a specific detector variant.

TABLE II: Clean performance scores of E-GraphSAGE on CTU-13 and TON-IoT datasets.

Dataset	Threat	<i>F1-score</i>		<i>Precision</i>		<i>Recall</i>	
		<i>Baseline</i>	<i>Hardened</i>	<i>Baseline</i>	<i>Hardened</i>	<i>Baseline</i>	<i>Hardened</i>
CTU-13	<i>Neris</i>	0.880	0.891	0.948	0.954	0.821	0.836
	<i>Rbot</i>	0.989	0.989	0.987	0.988	0.991	0.990
	<i>Virut</i>	0.945	0.945	0.979	0.947	0.913	0.944
	<i>Menti</i>	0.996	0.996	0.993	0.993	1.000	1.000
	<i>Murlo</i>	0.984	0.991	0.991	0.987	0.977	0.995
	μ (σ)	0.959 (0.048)	0.962 (0.045)	0.980 (0.018)	0.974 (0.022)	0.940 (0.075)	0.953 (0.069)
TON-IoT	<i>Backdoor</i>	0.997	0.997	0.998	0.996	0.995	0.998
	<i>DDoS</i>	0.992	0.991	0.993	0.990	0.990	0.991
	<i>DoS</i>	0.993	0.994	0.995	0.995	0.992	0.994
	<i>Injection</i>	0.991	0.994	0.991	0.989	0.992	1.000
	<i>Password</i>	0.994	0.992	0.995	0.987	0.994	0.996
	<i>Ransomware</i>	0.998	0.999	0.997	0.998	1.000	1.000
	<i>Scanning</i>	0.996	0.996	0.997	0.997	0.995	0.995
	<i>XSS</i>	0.987	0.995	0.993	0.990	0.981	1.000
	μ (σ)	0.994 (0.004)	0.995 (0.003)	0.995 (0.003)	0.993 (0.004)	0.992 (0.005)	0.997 (0.003)

We observe that the baseline classifiers, trained on clean data, are highly vulnerable to all types of attacks. This behavior aligns with the findings reported in [11], thereby empirically motivating our study and validating our experimental setup.

Across nearly all scenarios, **C2x_B** attacks prove to be the most damaging. This observation supports our decision to employ manipulated benign netflows during adversarial training, which enhances the model’s ability to recognize malicious patterns hidden within seemingly legitimate connections. By contrast, **C2x_M** attacks exhibit a different trend: the *DR* curve does not decline monotonically but often increases beyond a certain perturbation threshold. As noted in [11], this occurs because large numbers of malicious samples create topological patterns that are easier for the detector to identify. Conversely, **add node** attacks are the least effective. Indeed, they affect E-GraphSAGE performance only when attackers inject more than 100 new nodes within the graph.

Most importantly, our adversarial training framework yields substantial gains in robustness. This is evident from the slower decline in *DR* of the hardened detectors compared to the baseline models, particularly for the *Rbot*, *Menti*, *Murlo*, *DDoS*, and *Password* variants. In particular, for the *Menti* botnet under **C2x_B** attacks, the average *DR* across all perturbation steps rises from 0.027 of the baseline classifier to 0.926 of the hardened one, corresponding to an absolute improvement of nearly 90%.

Even under high-intensity perturbations, the hardened classifiers consistently outperform their baseline counterparts, validating our hypothesis that adversarial training with low-intensity perturbations effectively enhances GNN resilience to more severe structural attacks. However, none of the hardened classifiers achieve perfect detection, highlighting that there is still room for further improvement. Overall, augmenting the training data with structurally perturbed benign netflows proves to be a simple yet powerful strategy for strengthening

GNN detectors against structural adversarial attacks.

VII. CONCLUSION

GNN-based NIDS represent a significant advancement in cybersecurity, as they leverage graph-structured representations of network traffic to capture complex relational patterns that traditional approaches often miss. However, their dependence on graph topology also makes them uniquely vulnerable to structural adversarial attacks, a realistic threat that can undermine their reliability in practical deployments. Existing countermeasures often fall short due to unrealistic assumptions or prohibitive computational costs, leaving a critical gap in the defensive landscape.

In this paper, we introduced and validated a novel defense based on structural adversarial training specifically designed to harden GNN-based NIDS against such attacks. The core innovation of our framework lies in a lightweight yet effective mechanism for generating adversarial samples by selectively perturbing the source and destination hosts of benign netflows, thereby emulating realistic edge injection attacks. Our experimental campaign on CTU-13 and TON-IoT datasets, using E-GraphSAGE as the base GNN classifier, yielded two central findings: (1) our hardened detectors exhibit significantly improved resilience against diverse structural adversarial attacks and (2) this increased robustness is obtained without compromising (and in many scenarios even slightly improving) their performance on unperturbed inputs.

These results have significant implications for the deployment of GNN-based NIDS in operational environments. Our method offers an efficient and practical defense strategy that does not rely on pre-existing adversarial data or costly perturbation-generation procedures. By focusing on low-degree node replacement for benign flow perturbations, we demonstrate that adversarial training can meaningfully strengthen the resilience of GNN detectors against realistic structural threats.

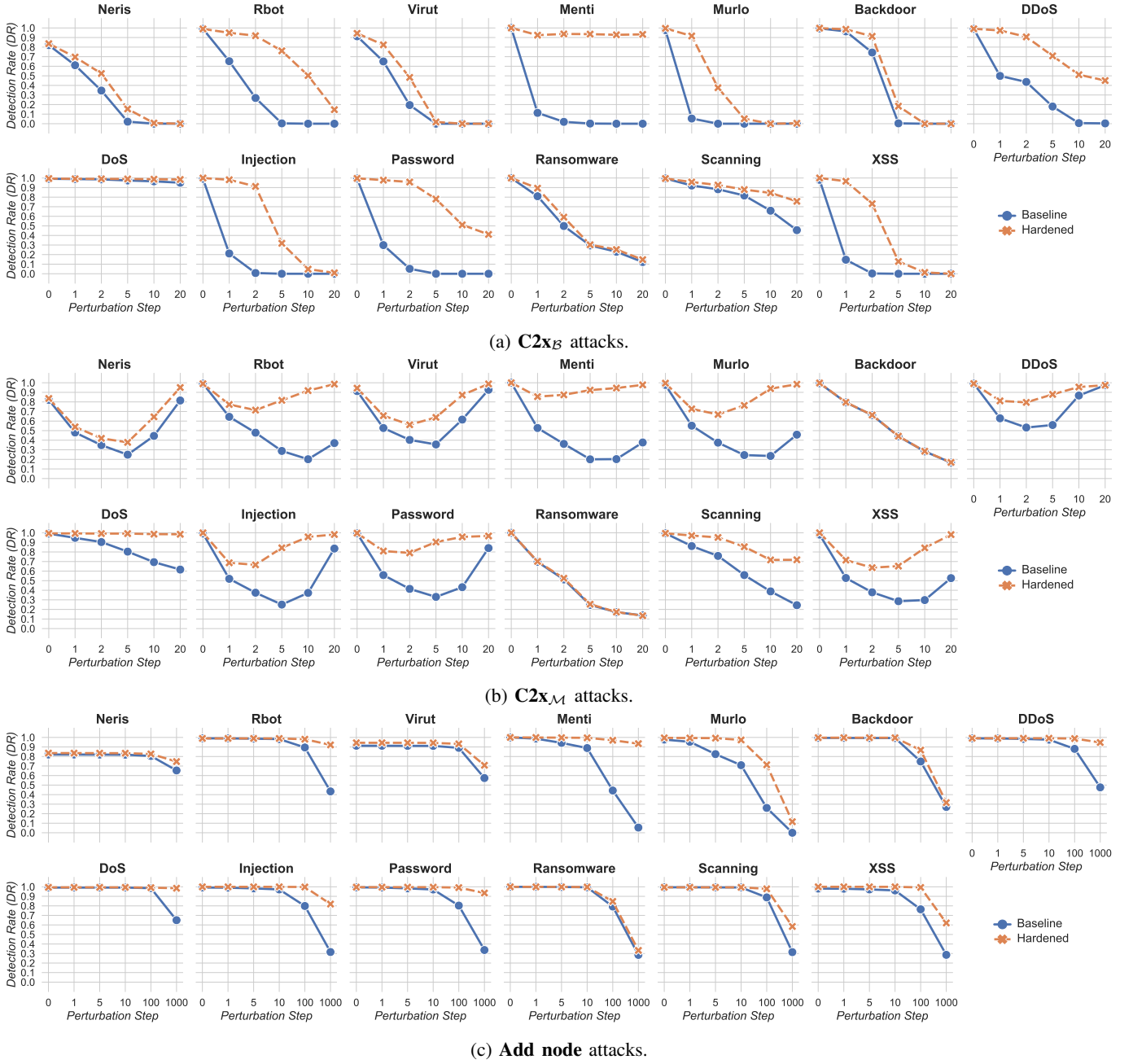


Fig. 3: Adversarial robustness scores of E-GraphSAGE against structural attacks.

ACKNOWLEDGMENT

This work has been supported by the project C4SI funded by the Emilia-Romagna region within the PR-FESR 2021-2027 (CUP E67G22000630003). This work has been supported by the European Union’s Horizon 2020 Research and Innovation Program under the project FALCON (Grant Agreement No. 101121281).

REFERENCES

- [1] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *2010 IEEE symposium on security and privacy*. IEEE, 2010, pp. 305–316.
- [2] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos, and F. Di Franco, “The role of machine learning in cybersecurity,” *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023.
- [3] G. Apruzzese, P. Laskov, and J. Schneider, “Sok: Pragmatic assessment of machine learning for network intrusion detection,” in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 592–614.
- [4] B. Biggio and F. Roli, “Wild patterns: Ten years after the rise of adversarial machine learning,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2154–2156.
- [5] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, “Intriguing properties of adversarial ml attacks in the problem space,” in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1332–1349.

- [6] W. Jiang, "Graph-based deep learning for communication networks: A survey," *Computer Communications*, vol. 185, pp. 40–54, 2022.
- [7] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-graphsage: A graph neural network based intrusion detection system for iot," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2022, pp. 1–9.
- [8] A. Venturi, M. Ferrari, M. Marchetti, and M. Colajanni, "Arganids: a novel network intrusion detection system based on adversarially regularized graph autoencoder," in *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 2023, pp. 1540–1548.
- [9] D. Pujol-Perich, J. Suárez-Varela, A. Cabellos-Aparicio, and P. Barlet-Ros, "Unveiling the potential of graph neural networks for robust intrusion detection," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 4, pp. 111–117, 2022.
- [10] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 2847–2856.
- [11] A. Venturi, D. Stabili, and M. Marchetti, "Problem space structural adversarial attacks for network intrusion detection systems based on graph neural networks," *arXiv preprint arXiv:2403.11830*, 2024.
- [12] A. Venturi, D. Pellegrini, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni *et al.*, "Practical evaluation of graph neural networks in network intrusion detection," in *CEUR Workshop Proceedings*, vol. 3488. CEUR-WS, 2023.
- [13] Z. Zhan, H. Zhou, and H. Haddadi, "Real-iot: Characterizing gnn intrusion detection robustness under practical adversarial attack," *arXiv preprint arXiv:2507.10836*, 2025.
- [14] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [15] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: an ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [16] A. Yehezkel, E. Elyashiv, and O. Soffer, "Network anomaly detection using transfer learning based on auto-encoders loss normalization," in *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*, 2021, pp. 61–71.
- [17] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021.
- [18] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.
- [19] X. Zhou, W. Liang, W. Li, K. Yan, S. Shimizu, and K. I.-K. Wang, "Hierarchical adversarial attacks against graph-neural-network-based iot network intrusion detection system," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9310–9319, 2021.
- [20] M. Zhong, M. Lin, C. Zhang, and Z. Xu, "A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges," *Computers & Security*, vol. 141, p. 103821, 2024.
- [21] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrđić, P. Laskov, G. Giacinto, and F. Roli, "Evasion attacks against machine learning at test time," in *Joint European conference on machine learning and knowledge discovery in databases*. Springer, 2013, pp. 387–402.
- [22] I. Corona, G. Giacinto, and F. Roli, "Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues," *Information sciences*, vol. 239, pp. 201–225, 2013.
- [23] G. Apruzzese and M. Colajanni, "Evading botnet detectors based on flows and random forest with adversarial samples," in *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*. IEEE, 2018, pp. 1–8.
- [24] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, and J. Yang, "Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3972–3987, 2022.
- [25] H. Hussain, T. Duricic, E. Lex, D. Helic, M. Strohmaier, and R. Kern, "Structack: Structure-based adversarial attacks on graph neural networks," in *Proceedings of the 32nd ACM Conference on Hypertext and Social Media*, 2021, pp. 111–120.
- [26] H. Wu, C. Wang, Y. Tyshetskiy, A. Docherty, K. Lu, and L. Zhu, "Adversarial examples on graph data: Deep insights into attack and defense," *arXiv preprint arXiv:1903.01610*, 2019.
- [27] N. Entezari, S. A. Al-Sayouri, A. Darvishzadeh, and E. E. Papalexakis, "All you need is low (rank) defending against adversarial attacks on graphs," in *Proceedings of the 13th international conference on web search and data mining*, 2020, pp. 169–177.
- [28] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, and J. Tang, "Graph structure learning for robust graph neural networks," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 66–74.
- [29] X. Zhang and M. Zitnik, "Gnn-guard: Defending graph neural networks against adversarial attacks," *Advances in neural information processing systems*, vol. 33, pp. 9263–9275, 2020.
- [30] D. Zhu, Z. Zhang, P. Cui, and W. Zhu, "Robust graph convolutional networks against adversarial attacks," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1399–1407.
- [31] K. Xu, H. Chen, S. Liu, P.-Y. Chen, T.-W. Weng, M. Hong, and X. Lin, "Topology attack and defense for graph neural networks: An optimization perspective," *arXiv preprint arXiv:1906.04214*, 2019.
- [32] K. Xu, S. Liu, P.-Y. Chen, M. Sun, C. Ding, B. Kailkhura, and X. Lin, "Towards an efficient and general framework of robust training for graph neural networks," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 8479–8483.
- [33] L. Gosch, S. Geisler, D. Sturm, B. Charpentier, D. Zügner, and S. Günnemann, "Adversarial training for graph neural networks: Pitfalls, solutions, and new directions," *Advances in neural information processing systems*, vol. 36, pp. 58 088–58 112, 2023.
- [34] G. Apruzzese, M. Andreolini, M. Marchetti, A. Venturi, and M. Colajanni, "Deep reinforcement adversarial learning against botnet evasion attacks," *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 1975–1987, 2020.
- [35] A. Venturi, D. Galli, D. Stabili, M. Marchetti *et al.*, "Hardening machine learning based network intrusion detection systems with synthetic net-flows," in *CEUR WORKSHOP PROCEEDINGS*, vol. 3731. CEUR-WS, 2024.
- [36] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, "Modeling realistic adversarial attacks against network intrusion detection systems," *Digital Threats: Research and Practice (DTRAP)*, vol. 3, no. 3, pp. 1–19, 2022.
- [37] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.
- [38] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [39] T. Pang, X. Yang, Y. Dong, H. Su, and J. Zhu, "Bag of tricks for adversarial training," *arXiv preprint arXiv:2010.00467*, 2020.
- [40] H. Bostani, J. Cortellazzi, D. Arp, F. Pierazzi, V. Moonsamy, and L. Cavallaro, "On the effectiveness of adversarial training on malware classifiers," *arXiv preprint arXiv:2412.18218*, 2024.
- [41] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *computers & security*, vol. 45, pp. 100–123, 2014.
- [42] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems," *Ieee Access*, vol. 8, pp. 165 130–165 150, 2020.
- [43] D. Galli, A. Venturi, I. Marasco, M. Marchetti *et al.*, "Evaluating explainability of graph neural networks for network intrusion detection with structural attacks," in *CEUR WORKSHOP PROCEEDINGS*, vol. 3962. CEUR-WS, 2025.
- [44] A. Venturi, G. Apruzzese, M. Andreolini, M. Colajanni, and M. Marchetti, "Drelab-deep reinforcement learning adversarial botnet: A benchmark dataset for adversarial attacks against botnet intrusion detection systems," *Data in Brief*, vol. 34, p. 106631, 2021.
- [45] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.